



Guide pas à pas — iFlow SAP CPI de création de contacts

Ce guide configure le flux HTTP synchrone suivant : le client appelle CPI, CPI appelle le service OData SAP via le Cloud Connector, puis CPI retourne immédiatement la réponse JSON au client.

Client HTTP

- HTTPS Sender
- Groovy : Process Input Data
- Router —(payload invalide)—→ End Message
 - └─(payload valide)—→ Request Reply → OData V2 Receiver (SAP Gateway)
- Groovy : Response Handler
- réponse HTTP au client

Exception Subprocess

- Groovy : Error Handler → réponse HTTP 500 au client

Composants CPI à placer

Dans l'éditeur de l'iFlow, placez exactement les composants suivants :

Zone CPI	Composant	Quantité	Rôle
Participant expéditeur	HTTPS Sender	1	Reçoit le POST du client.
Integration Process	Start Event	1	Point de départ du processus principal.
Integration Process	Groovy Script	2	Prépare la requête puis normalise la réponse SAP.
Integration Process	Router	1	Arrête les payloads invalides avant l'appel SAP.
Integration	Request	1	Appel synchrone vers SAP Gateway.

Zone CPI	Composant	Quantité	Rôle
Process	Reply		
Participant récepteur	OData V2 Receiver	1	Appelle ContactSet dans SAP via le Cloud Connector.
Integration Process	End Message	2	Termine respectivement le chemin fonctionnel et le chemin HTTP 400.
Exception Subprocess	Exception Start Event	1	Capture les exceptions techniques.
Exception Subprocess	Groovy Script	1	Formate l'erreur technique.
Exception Subprocess	End Message	1	Retourne l'erreur HTTP 500.

1. Prérequis

Avant de créer l'iFlow :

- le service Gateway `/sap/opu/odata/sap/ZCONTACTS_SRV` est activé et son **ContactSet** est disponible ;
- le Cloud Connector expose ce chemin vers le système SAP on-premise ;
- le compte technique SAP possède les autorisations nécessaires aux BAPIs Business Partner ;
- les scripts du dépôt sont disponibles :
 - [groovy_process_data.groovy](#)
 - [groovy_response_handler.groovy](#)
 - [groovy_error_handler.groovy](#)

2. Créer l'artefact CPI

1. Dans **Design**, créez ou ouvrez le package d'intégration voulu.
2. Cliquez sur **Add** → **Integration Flow**.

3. Donnez-lui un nom, par exemple `IFLOW_CREATE_BP_CONTACT`.
4. Ouvrez l'iFlow en mode **Edit**.
5. Conservez le **Start Event** créé automatiquement dans l'Integration Process.

3. Configurer l'entrée HTTPS

1. Ajoutez un adaptateur **HTTPS Sender** au point de départ de l'iFlow.
2. Dans l'onglet **Connection**, définissez l'adresse :

```
/v1/sap/contacts
```

3. Configurez l'authentification et le rôle autorisé selon les standards du tenant (par exemple `ESBMessaging.send`).
4. Le client appellera alors :

```
https://<tenant-cpi>/http/v1/sap/contacts
```

5. Reliez l'adaptateur HTTPS au **Start Event** du processus principal.

4. Ajouter le script de préparation

1. Ajoutez une étape **Groovy Script** juste après le HTTPS Sender.
2. Dans **Resources**, ajoutez [groovy_process_data.groovy](#).
3. Sélectionnez ce script dans l'étape Groovy.

Le script :

- valide `BpParent`, `FirstName`, `LastName` et `Country` lorsqu'une adresse est renseignée ;
- complète les valeurs par défaut `BpCategory: 1`, `Grouping: ZC` et `BpRole: BUP001` si elles sont absentes ;
- normalise `Country` en majuscules et complète `DateFrom` si nécessaire ;
- retourne `HTTP 400` avec `StatusCode: ERROR` pour un payload technique invalide.

5. Ajouter le Router de validation

1. Ajoutez un composant **Router** après le script de préparation.

2. Créez une première branche avec une condition **Non-XML** :

```
${property.is_integration_failed} = 'true'
```

3. Reliez cette branche à un **End Message**. Elle renvoie le body et le code HTTP 400 déjà préparés par `groovy_process_data.groovy`.
4. Définissez l'autre branche comme **Default Route**. Seuls les payloads valides l'empruntent vers SAP.

6. Configurer l'appel OData vers SAP

1. Ajoutez une étape **Request Reply** sur la **Default Route** du Router.
2. Ajoutez un participant récepteur puis reliez le canal de réception du **Request Reply** à son adaptateur **OData V2 Receiver**.
3. Configurez l'onglet **Connection** de l'adaptateur :

Paramètre	Valeur
Address	<code>https://<virtual-host>:<port>/sap/opu/odata/sap/ZCONTACTS_SRV</code>
Proxy Type	<code>On-Premise</code>
Location ID	Renseigner seulement si le Cloud Connector en utilise un
Authentication	Compte technique SAP (sans enregistrer le mot de passe dans le modèle)

4. Dans l'onglet de traitement de l'adaptateur, définissez :

Paramètre	Valeur
Resource Path	<code>ContactSet</code>
Operation	<code>CREATE</code>
Request format	<code>JSON</code>
Response format	<code>JSON</code>

5. Reliez l'adaptateur OData au **Request Reply**.

Aucun **Content Modifier** n'est requis dans ce flux : les scripts Groovy préparent le body, les

propriétés et les headers nécessaires.

7. Normaliser la réponse fonctionnelle

1. Ajoutez une seconde étape **Groovy Script** après le **Request Reply**, sur le chemin nominal.
2. Ajoutez `groovy_response_handler.groovy` dans les ressources et sélectionnez-le dans cette étape.

Ce script retire l'enveloppe OData V2 si présente et retourne le JSON de l'entité `Contact`. Il positionne `HTTP 200` pour les retours fonctionnels `SUCCESS`, `EXISTS` ou `ERROR`.

8. Configurer le sous-processus d'exception

1. Ajoutez un **Exception Subprocess** dans l'iFlow ; il n'est pas relié au chemin nominal.
2. Dans ce sous-processus, ajoutez une étape **Groovy Script**.
3. Ajoutez `groovy_error_handler.groovy` dans les ressources et associez-le à l'étape.
4. Terminez le sous-processus avec un **End Message**.

Une indisponibilité réseau, une erreur d'authentification ou une exception SAP technique est alors retournée au client avec `HTTP 500` et `StatusCode: ERROR`.

9. Vérifier le dessin final

Le chemin principal doit respecter cet ordre :

1. HTTPS Sender
2. Groovy `groovy_process_data.groovy`
3. Router
 - branche `${property.is_integration_failed} = 'true'` → End Message
 - Default Route → Request Reply
4. OData V2 Receiver (rattaché au Request Reply)
5. Groovy `groovy_response_handler.groovy`
6. End Message

Le sous-processus d'exception contient uniquement le script `groovy_error_handler.groovy`, puis son `End Message`.

10. Déployer et tester

1. Sauvegardez l'iFlow puis cliquez sur **Deploy**.
2. Dans Postman, envoyez une requête **POST** vers l'URL HTTPS CPI avec le header **Content-Type: application/json**.
3. Utilisez [postman_payload_create.json](#) comme corps de requête.
4. Vérifiez les résultats :

Cas	HTTP	StatusCode
Création réalisée	200	SUCCESS
Même prénom/nom déjà actif pour le même parent	200	EXISTS
BP parent absent	200	ERROR
JSON ou champs techniques invalides	400	ERROR
Erreur réseau ou exception SAP technique	500	ERROR

5. En cas d'erreur, activez temporairement le niveau de trace dans **Monitor → Integrations and APIs** puis consultez le journal du message. Désactivez-le après le diagnostic.